

AI 编码代理测试策略

横纵分析法深度研究报告

研究时间：2026-08-08 | 研究对象：AI 编码代理的测试策略 | 研究对象类型：工程方法与产品机制

作者: Codex

研究时间：2026-08-08 | 研究对象：AI 编码代理的测试策略 | 研究对象类型：工程方法与产品机制

一句话定义

“测试狂魔病”不是测试太多这个单一问题，而是验证工作没有被分层、筛选和限时：模型把每一次改动都当成一次完整交付，于是不断生成测试、不断运行全量套件、不断根据新失败继续扩张。成熟团队解决的不是“劝模型少写”，而是让验证路径变成一条有风险等级、有反馈速度、有预算和有升级出口的流水线。

结论摘要

我的判断是：对 AI 编码代理，最稳的默认值不是 `off`，也不是 TDD，而是“先验证能运行，再决定是否新增测试”。测试只在三个条件之一成立时升级：修复了已知缺陷、改变了稳定的公共行为、或改动触及高风险不变量。

成熟案例反复出现五个共同机制：

1. 按测试属性管理，而不是按测试数量管理。Google 2024 年把测试权衡概括为 SMURF：Speed、Maintainability、Utilization、Reliability、Fidelity。测试多不等于质量高；慢、脆、不可定位的测试会直接减少交付能力。
2. 把快速检查和深度测试拆开。Aider 的官方配置把 lint 设为自动，而 `--auto-test` 默认关闭；测试可以显式执行，也可以由用户打开“每次编辑后运行”。这正是 Agent 默认行为应该采用的边界。
3. 只测受影响的范围。Google 的 mutation testing 服务只对变更行生成突变，并用覆盖信息找出能杀死突变的最小测试集合；大型仓库不把全量测试塞进每次编辑的反馈环。
4. 测试必须能解释失败，并且只验证行为。Google 明确反对 change-detector tests：复制实现、验证调用顺序、给 getter/setter 补覆盖率，都会让重构变慢却不增加缺陷发现能力。
5. 规则、工具和流程要叠加。GitHub Copilot 依赖仓库内的 `AGENTS.md` / `copilot-instructions.md` 描述构建与验证路径；Claude Code 的权限和 `PreToolUse` hook 才是真正能拦截工具调用的产品级边界；OpenAI Codex 自身的仓库规则则把“不要给静态值写测试”“优先跑受影响项目”“全量测试需要额外确认”写成具体工程约束。

所以，解决方案不是“少写测试”四个字，而是下面这个状态机：

```

classify change
|- docs/style/one-off script -> compile or run check; no new tests by default
|- UI/glue/business wiring -> affected tests + smoke; add one behavior test if needed
|- shared logic/state/data -> behavior tests + targeted integration tests
|- auth/payment/race/migrate -> regression + integration/E2E + human confirmation

budget exhausted or failure cannot be isolated
`-> stop expanding; report evidence, cause, and remaining risk

```

一、纵向分析：测试从“越多越安心”走到“反馈价值最大化”

1. 早期自动化：测试的价值首先是反馈速度

单元测试、集成测试和端到端测试并不是同一件事的不同规模版本。它们在反馈速度、隔离能力、维护成本、资源使用和生产相似度上存在结构性差异。

Google Testing Blog 的《Test Sizes》把测试按 small、medium、large 分层，并用约束描述它们：small 测试不访问网络和外部系统，时间上限约 60 秒；medium 允许更多本地资源，时间上限约 300 秒；large 可以使用外部系统、多线程和 sleep，时间上限约 900 秒以上。这个分类的关键不在数字本身，而在于测试大小是一个工程契约：越大的测试越不能被当成每次编辑后的即时反馈。

Google 随后在端到端测试的讨论中把问题说得更直接：失败测试本身不产生用户价值，真正产生价值的是修复缺陷；一个好的反馈环必须“快、可靠、能隔离失败”。端到端套件即使抓到了真实缺陷，也可能因为环境、合作方和 flaky test 让开发者一天都无法知道修复是否有效。

这对 Agent 很重要。模型会把“运行测试”当作一个可重复的工具动作；如果工程没有规定反馈时限，它就会在失败后反复重试。成熟团队的做法是把“能否快速归因”放在“覆盖了多少场景”之前。

2. TDD 与测试金字塔：从工作法变成组合原则

TDD 的红-绿-重构循环适合需要先澄清行为、且反馈极快的核心逻辑。问题出在把 TDD 当成所有任务的统一入口：改一个配置、写一次性导出脚本、调整 CSS、接一层 API 胶水，也被强行拆成测试先行、实现、重构、再扩展测试。

Mike Cohn 的测试金字塔和 Martin Fowler 的实践化解释，真正留下的是两个原则：测试应有不同粒度；越靠近真实系统、越高层、越慢的测试，数量越少。Fowler 特别提醒，测试冰淇淋锥会造成维护噩梦；私有方法属于实现细节，不应因为追求覆盖率而单独测试；公共接口和用户可观察行为才是稳定的测试边界。

因此，测试金字塔不是“单测必须占 80%”的配额表，更不是 Agent 的任务清单。它是一种成本结构：把大多数反馈放在快速、隔离、可定位的层，把少数高保真验证放在集成和 E2E 层。

3. 持续交付：质量门禁必须服务于流速

持续交付把测试放进流水线，但并没有要求每个开发动作都运行完整流水线。成熟流程通常分为：

- 编辑级反馈：类型检查、编译、格式化、lint、极窄的相关单测。
- 提交级门禁：受影响目标的单测/集成测试、静态分析、构建产物检查。
- 合并或发布级验证：全量测试、跨平台测试、E2E、性能和安全检查。
- 合并后的异步深测：mutation testing、全量覆盖率、长时间稳定性、真实环境回放。

如果把第四层塞到第一层，开发者会等待；如果删掉第四层，团队会失去对系统性风险的观察。成熟做法是保留深测，但让它异步、增量、可追踪，而不是让每个 Agent 回合都承担全部成本。

这条路线背后还有一条经常被忽略的学术脉络。1975 年 Goodenough 与 Gerhart 已经把问题从“测试越多越好”改写为测试选择的 reliability 与 validity；1997 年 Rothermel 与 Harrold 正式提出 regression test selection，只运行会触达变更代码的既有测试；2006 年 time-aware test prioritization 进一步承认 nightly/build 有固定时间窗，目标是在预算内尽早发现更多故障。今天大型团队的 affected-test selection，并不是偷工减料，而是几十年测试研究自然落到工程系统后的结果。

Google 2024 年关于小型 pull request 的文章也把迭代速度与质量连接起来：小而聚焦的变更更容易审查、失败更容易定位、出错后更容易回滚。对 Agent 来说，这意味着“限制任务 diff 的范围”往往比“限制测试文件数”更有效。一个 20 行业务改动配一个高价值回归测试，通常比一个 200 行混杂重构和十几个测试文件的 PR 更安全。

4. 近年的转向：从覆盖率驱动到测试组合优化

覆盖率适合找“完全没有被测试触达的区域”，不适合直接当成每个任务的成功目标。Google 2024 年发布的 SMURF 框架，把测试组合的权衡从单一金字塔扩展为五个维度：

维度	对 Agent 的含义	典型失败表现
Speed	反馈是否足够快	改一行代码跑十分钟全量测试
Maintainability	测试是否能随需求演进	实现一变就大面积改测试
Utilization	CPU、内存、数据库等资源是否合理	测试数量增长导致 CI 超线性变慢
Reliability	是否只在真实问题出现时失败	flaky test 触发无意义重试
Fidelity	是否接近真实运行环境	单测全绿但真实集成失败

SMURF 的价值在于承认取舍：单测通常速度、维护性、资源利用率和可靠性更好，但保真度较低；E2E 保真度高，却在其他维度成本更大。Agent 选择测试层级时，应该先问“这次改动需要哪种证据”，而不是机械复制同一种测试。

5. 2020 年代的成熟信号：测试本身也要被审计

Google 的 Mutation Testing Service 是一个非常具有代表性的案例。它最初发现，直接给开发者展示所有突变会产生海量噪声，其中大量是“unproductive mutants”：为它们补测试只会制造脆弱的 change-detector tests。后来系统加入了多层过滤：忽略 arid nodes；只看覆盖到的变更行；每行最多报告一个突变；每个文件最多报告七个突变；用覆盖信息运行能杀死突变的最小测试集合；服务默认 opt-in，并让作者反馈“Not useful”，把反馈反过来改进启发式规则。

这不是单纯的 mutation testing 技术故事，而是一套非常适合 Agent 的产品原则：

质量工具的输出也必须有预算；不能把所有可能的发现都扔给开发者；被拒绝的建议必须成为下一轮过滤规则。

Meta 的 Predictive Test Selection 则把“预算内选测”做成了大规模生产系统。其公开论文描述了三层兜底：diff-time 自动运行预测子集，land-time 运行更全面的子集，后台 stabilization 每数小时执行全量测试；release candidate 只能基于全绿快照。论文报告的生产结果包括：执行测试量低于依赖图方案的三分之一，基础设施成本约减半，同时捕获超过 95% 的单测失败和超过 99.9% 的有缺陷 diff。重点不是机器学习本身，而是系统没有假装快速层能做到绝对完整，它用后置全量验证承担剩余风险。

二、横向分析：成熟团队与工具到底怎么做

案例 A：Google - 用测试大小、可行动失败和增量分析控制规模

Google 的实践不是“所有代码都 TDD”，而是把测试当成有不同反馈特征的资产。

1. Test Sizes 规定了 small/medium/large 的资源与时长边界。
2. 端到端测试文章强调反馈环必须快、可靠、隔离失败；失败测试只是发现问题，bug fix 才是价值落点。
3. SMURF 让团队在测试速度、维护性、资源、可靠性和保真度之间做组合决策。
4. Mutation Testing Service 只分析变更行、最小化测试执行、抑制低价值突变，并限制每行/每文件的报告噪声。
5. Small Pull Requests 把变更控制在一个清晰目的内，降低审查和回滚成本。

对 Agent 的可迁移做法是：

- 用测试“大小”和“反馈时间”作为硬门槛，不把 E2E 当即时反馈。
- 用“失败是否可行动”评价测试，不用断言数量评价测试。
- 对变更做 affected-test selection，测试全量放到提交后或异步流水线。
- 对测试建议做采样、去重和反馈学习，宁可少报高价值问题，不要报全量可能性。

案例 B：OpenAI Codex 仓库 - 把“不要测什么”和“测到哪一层”写进仓库规则

OpenAI Codex 官方仓库的 `AGENTS.md` 是一个很实用的反例：它没有写“每行都要测”，而是写了多个可判定边界。

- 静态定义的值不增加测试。
- 已删除逻辑不补负向测试。
- Agent 行为变化优先使用集成测试，而不是堆内部单元测试。
- 先跑变更项目的专项测试；涉及公共模块时再扩大范围。
- 完整测试套件需要额外确认，避免日常开发默认承担全量成本。
- 变更规模有约 800 行的工程性警戒线，复杂逻辑倾向拆成更小阶段。

这套规则解决了两个经常被混淆的问题：要不要测试和测试怎么跑。前者由风险和公共行为决定，后者由受影响项目和执行成本决定。它还把“Agent 逻辑变更必须补集成测试”限制在真正影响用户行为的场景，而不是给所有代码加单测。

案例 C：Aider - 默认自动 lint，默认不自动测试

Aider 官方文档提供了一个与用户痛点高度吻合的产品级案例：

- `--lint-cmd` 可指定 lint 命令；`--auto-lint` 默认开启。
- `--test-cmd` 只声明测试命令。
- `--auto-test` 默认关闭；用户可以用 `/test <command>` 显式运行，也可以按项目需要开启每次编辑后测试。
- 测试失败后，Aider 会把标准输出/错误和非零退出码交给模型修复。

这个默认值很有启发性：lint 和编译通常是低成本、高信号的即时反馈；测试可能涉及数据库、网络 and 长时间构建，因此不应该默认在每次模型编辑后运行。Aider 没有禁止模型写测试，而是把“自动执行测试”从默认动作降级为项目配置。

适合直接借鉴的 Agent 默认流程是：

```
after each edit: format / lint / typecheck / compile
when risk matches or user asks: targeted tests
before commit: one affected-test pass
after merge: full / E2E / mutation / performance / soak
```

案例 D：Claude Code - 规则塑造意图，权限和 hook 负责硬拦截

Claude Code 官方文档明确区分了两件事：`CLAUDE.md` 之类的指令文件只影响模型“想做什么”，权限规则由产品执行，决定“允许做什么”。权限支持 `allow`、`ask`、`deny`，并且 `deny` 优先级最高；`PreToolUse` hook 在工具调用前触发，可以返回 `permissionDecision: "deny"` 阻止调用。

这提供了三层落地方式：

1. 在项目指令中写测试策略，减少模型主动扩张。
2. 对危险或高成本测试命令做 Bash `deny/ask`，例如禁止没有目标参数的全量测试命令。
3. 用 `PreToolUse` hook 读取工具调用、统计本任务测试执行次数或检查目标路径，超过预算直接阻断。

但不建议把所有 `*.test.*` 文件一刀切 `deny`。官方权限语法对工具和路径有严格匹配规则，粗糙的 glob 容易误伤高风险回归修复；更好的拦截对象是“无目标的全量执行”和“重复重试”，而不是测试文件这个扩展名。

案例 E：GitHub Copilot - 用仓库上下文减少探索和错误尝试

GitHub 官方文档支持三种层级的自定义指令：仓库级 `copilot-instructions.md`、路径级 `instructions`，以及按目录就近生效的 `AGENTS.md`。文档建议把 `bootstrap`、`build`、`test`、`run`、`lint` 和工具版本写清楚，目标是减少 Agent 的搜索、命令失败和 CI 退回。

它没有把“多写测试”当作质量保证，而是把验证命令、项目结构和路径规则作为上下文契约。对测试狂魔病，最直接的启示是：模型若不知道“哪个测试最相关、完整套件怎么跑、哪些目录是一次性脚本”，就会用写更多测试来弥补不确定性。把这些信息写进就近的策略文件，往往比在全局提示词里重复劝告有效。

案例 F：Martin Fowler / Thoughtworks - 测试金字塔的真正约束是成本结构

Fowler 的实践化测试金字塔并不要求固定比例，而是强调两个约束：测试有不同粒度；越高层越少。它明确反对为了 100% 覆盖率而测试私有方法、getter/setter 或实现细节，并建议用快速单测覆盖大量局部行为，用少数服务/集成/E2E 测试确认边界和真实协作。

这套思想适合做 Agent 的“判断规则”，而不是写成配额：

- 先测公共接口和业务不变量。

- 能用窄集成测试证明的，不要复制成大量内部 mock 单测。
- 能用一条回归测试锁住的，不要为同一根因铺十个近似用例。
- 测试若经常因为无关实现变化而失败，优先改善测试 API 和断言表达，而不是继续补测试。

案例 G：Meta - 快速层允许选测，完整性由 **stabilization** 兜底

Meta 的公开流程把测试反馈拆成三个时间尺度：开发 diff 阶段目标反馈在十分钟内；代码准备落地时跑更全面的选择集；后台每数小时跑全量 **stabilization**。预测模型用历史失败、代码和测试关系估计风险，并按目标 **ChangeRecall** 调整阈值；**flaky failure** 被单独建模，避免把不稳定噪声当作真实回归。

这套方案对普通仓库不要求复制机器学习。小型项目可以用目录、依赖图、测试标签和最近失败记录完成八成效果。真正该复制的是契约：快速层有明确 SLA，允许保守选测；后台全量必须持续存在；漏检风险需要被测量，而不是靠“我觉得这些测试够了”。

案例 H：Microsoft、GitLab 与 Kubernetes - 选测必须带保守 **fallback**

Microsoft Azure DevOps 的 Test Impact Analysis 不只运行静态判断出的 impacted tests，还纳入历史失败和新增测试；遇到无法理解的文件类型时退回全量，并允许配置周期性全量验证。这说明成熟的 test impact analysis 必须有安全阀，不能把“未识别”解释成“不需要测试”。

GitLab 用 `rules:changes` 只把相关 job 加入 pipeline，再用 merge trains 验证多个待合并变更组合后的结果。它对 flaky 测试允许独立进程重试一次；若持续阻塞主分支，则 quarantine，但隔离不是删除责任，仍要保留治理记录。

Kubernetes 的 Prow 将 job 分为 presubmit、postsubmit 和 periodic，并支持 `run_if_changed`、`skip_if_only_changed`。新增 job 不会一上来就变成 required gate，而是先手动运行、观察若干次，再自动运行并经过数天稳定性观察后接入门禁。这个 rollout 方式尤其适合 Agent 新增测试：先证明测试稳定和有信号，再让它阻塞所有人的合并。

案例 I：Google flaky 治理 - 隔离不稳定测试，但不把隔离当修复

Google 公开数据曾显示，约 1.5% 的测试运行结果是 flaky，约 16% 的测试至少出现过一次 flaky。其系统允许失败重跑，对高频 flaky 测试自动 quarantine 并创建缺陷跟踪。官方同时警告，隔离可能掩盖真实 race condition，因此 quarantine 只是保护主反馈环的临时措施，必须有 owner 和修复责任。

这给 Agent 一个非常明确的边界：同一测试第二次通过不能自动判定代码正确；应该把结果标成“疑似 flaky”，停止继续修改生产代码或测试，并把测试从即时门禁转交给专门治理流程。

三、成熟案例的共同模式

1. 质量边界不是覆盖率，而是“可证明的不变量”

一个测试值得存在，至少应该证明以下某一项：

- 对外接口在关键输入下返回正确结果。
- 已发生的缺陷不会回归。
- 模块边界的协议、序列化、权限或错误处理保持不变。
- 一个高风险状态转换满足业务不变量。
- 真实依赖之间的契约没有断裂。

如果测试只证明“这个函数调用了那个函数”“这个常量等于它自己”“getter 能返回字段原值”，它通常属于 change detector。Google 对这种测试的结论很硬：它不捕获缺陷，却增加维护成本，应该重写或删除。

2. 最成熟的“预算”是多维的

单任务“最多 3 个测试文件”可以作为 Agent 的保护阀，但不够成熟。更合理的预算至少包括：

预算维度	默认建议	超限动作
新增测试文件	0-1 个	停止写入，说明为何需要升级
新增测试代码	不超过生产 diff 的 50%-100%，视风险调整	检查是否在重复覆盖
即时测试时长	30-90 秒	改跑目标测试或只做编译/lint
测试执行次数	1 次，失败后最多定向重试 1 次	不允许盲目重复全量
全量测试	提交/合并阶段一次	从编辑回路移出
E2E/外部依赖	仅风险命中或发布前	异步执行并记录结果
Agent 回合	2 次失败仍无法归因	停止修测试，汇报失败证据

对于大型仓库，还应增加一个“漏检预算”：快速层允许少量风险转移到后置全量，但必须用历史数据验证选测策略，例如 change recall、post-merge escape rate 和因漏跑测试导致的回滚次数。Meta 用 ChangeRecall 约束预测选测，Microsoft 用无法分析时的全量 fallback，都是在管理漏检预算。

这里的“新增测试代码比例”不是质量目标，只是发现模型是否正在把任务改造成测试工程。如果业务改动只有 20 行，却生成 300 行测试，Agent 必须解释新增场景对应的公共不变量；解释不清就回退到最小验证。

3. 测试稳定性是硬质量指标

Google 的 SMURF 把 Reliability 单列出来，并指出 flaky test 会导致重复运行、浪费开发者时间和资源；Google 的“Test failures should be actionable”进一步要求：只看测试名和失败信息，就应该能开始调查，不必先加日志再重跑。

因此，项目不能只记录“测试通过率”，还应记录：

- 失败后首次定位所需时间。
- flaky 重试率和被隔离的测试数量。
- 失败是否能指向变更范围。
- 测试维护 PR 占比。
- 测试被删除/重写的原因。

一个经常失败但没人信的测试，比少一个测试更危险，因为它会训练 Agent 和人类一起忽略红灯。

4. 变更范围比测试数量更值得限制

Google 小型 PR 的经验说明，聚焦的变更更容易审查、定位和回滚。对 Agent，建议把任务拆成以下三种可独立交付的变更：

1. 行为改动：生产代码 + 必要的行为/回归测试。
2. 测试基础设施：测试工具、夹具、运行器、隔离和并发设置。
3. 重构/迁移：保持行为不变，主要依赖编译、既有测试和少数关键集成验证。

把三类东西塞进一个任务，模型很容易通过新增测试来掩盖范围失控。任务边界清晰后，测试数量自然会下降，因为每个测试都必须对当前行为改动负责。

四、交叉洞察：如何把这些案例变成 Agent 的默认 workflow

1. 推荐的四档测试强度

off：不新增测试，只做可运行性验证

适用：文档、配置、样式、一次性脚本、原型、纯重命名、静态值调整、无行为变化的机械重构。

必做：格式检查、类型检查或编译；脚本可以执行时跑一次最小 happy path；说明未新增测试的原因。

smoke：一条可观察行为检查

适用：UI、胶水层、简单 API 接线、导入导出、CLI 参数。

必做：一个最短运行路径或现有 smoke；只有发现回归风险时才新增测试文件。

standard：少量行为测试 + 受影响测试

适用：共享业务逻辑、状态转换、数据清洗、错误处理、可复用库。

必做：覆盖关键成功路径、一个高价值边界或回归；优先放在现有测试文件或现有测试框架中；只运行受影响测试。

thorough：完整验证与人工确认

适用：认证授权、支付结算、数据迁移、并发、持久化格式、外部协议、安全边界。

必做：行为测试、集成/契约测试、必要的 E2E 或真实依赖验证；提交前可以跑全量；Agent 不能在失败后自主扩大范围而不汇报。

2. 推荐的 Agent 状态机

```
S0 read repository policy, entry points, and existing verification commands
|
S1 classify change risk and affected directories
|
S2 run compile / typecheck / lint / smoke first
|
S3 discover affected tests; decide whether a regression test is necessary
|
S4 run targeted tests once
|- pass -> inspect diff, report evidence, finish
|- attributable failure -> fix production code, retry once
`- unclear failure -> stop adding tests; report env/flake/dependency issue
|
S5 upgrade to integration/E2E/full only for high risk or explicit request
```

关键点是 S4 的停止规则。许多 Agent 之所以“卡”，不是因为第一次测试失败，而是因为没有“失败后不再扩张”的状态。它们会继续添加边界用例、mock、fixture，再把由测试本身引入的失败当成新任务。

3. 可直接放进 AGENTS.md 的模板

```
## Testing policy

- Tests are not the default deliverable. First validate the requested behavior with the repository's existing build, typecheck, lint, smoke, or targeted test commands.
- Do not add tests for documentation, styling, static values, one-off scripts, prototypes, mechanical renames, or removed logic unless the user explicitly asks for them.
- Add a test only when at least one is true: this is a regression fix; a stable public behavior changed; a shared invariant/state transition is at risk; or the task is security/payment/persistence/concurrency sensitive.
- Test observable behavior and public contracts. Do not test private methods, getters/setters, call order, internal mocks, or implementation details unless they are themselves the contract.
- Default verification level is `smoke` for glue/UI/scripts and `standard` for reusable business logic. Use `thorough` only for high-risk changes or when requested.
- Per task budget: at most 1 new test file, 2 test executions, and 90 seconds of immediate test time. Full-suite/E2E tests belong to the commit or CI stage, not every edit loop.
- After one targeted failure, fix the production issue if the cause is clear. After one retry, stop and report the exact command, failure, suspected cause, and remaining risk. Do not keep adding tests to make an unclear failure disappear.
- Prefer existing test files, fixtures, helpers, and affected-test commands. Keep the change focused; separate test-infrastructure work from feature work.
- Before finishing, report: files changed, validation commands run, results, and tests not run.
```

4. 产品层面真正值得做的功能

如果要把这个问题做成 maka-agent 或其他 Agent 产品的能力，优先级应是：

1. 测试策略档位：`off / smoke / standard / thorough`，支持仓库级、目录级和任务级覆盖。
2. 受影响测试发现：从 git diff、依赖图、测试标签、覆盖信息和历史失败记录推断目标测试。
3. 双预算执行器：分别限制模型新增测试、测试运行次数、即时耗时和全量 CI 消耗。
4. 停止与升级状态机：失败一次看原因，失败两次停止；高风险改动可升级，但必须向用户说明。
5. 低价值测试检测：识别静态值测试、空断言、实现调用顺序、过度 mock、快照复制和重复场景。
6. 反馈学习：用户标记“无用”的测试建议或 mutation finding 后，记录到仓库级策略；不要只写进模型记忆。
7. 验证账本：在最终消息中列出运行过的命令、耗时、结果和未运行项目，避免“测试通过”变成无证据口号。
8. 渐进门禁：新测试先以 non-blocking/观察模式运行，证明稳定后才升级为 required；参考 Prow 的 job rollout，而不是让 Agent 一次提交就永久增加阻塞成本。

不建议第一版就做一个“禁止写测试”的总开关。它会在脚本目录看起来很有效，却会误伤真正需要回归测试的支付、权限和迁移代码。产品应该拦截的是高成本行为和低价值模式，并允许按风险升级。

五、给当前开发环境的落地顺序

第一天：先解决 Agent 的默认行为

1. 在仓库根目录和高频原型目录分别放 `AGENTS.md`，使用就近策略覆盖。
2. 明确 `build`、`typecheck`、`lint`、`smoke`、`targeted test`、`full test` 命令，给每个命令标注预计时长。
3. 关闭 TDD/superpowers 类 skill 的全局默认；只在核心库或高风险任务中显式启用。
4. 把“先跑现有测试，只有必要时新增测试”放在任务完成标准前面。

第二天：给测试命令包一层确定性脚本

不要让 Agent 自由拼接测试命令。为项目提供固定入口，例如：

```
./scripts/verify.sh fast      # format + lint + typecheck + compile
./scripts/verify.sh affected  # affected tests only
./scripts/verify.sh full      # full suite for CI/merge stage
```

脚本负责记录开始时间、测试次数、目标范围和退出码；如果检测到全量测试被放进编辑回路，直接返回“请使用 affected 或 full 阶段”的清晰错误。这样模型收到的是可行动提示，不会继续猜命令。

第三天：只对高成本行为加硬约束

在支持 hook/permission 的 Agent 中，优先拦截：

- 没有目标参数的全量测试。
- 同一个测试命令的第三次执行。
- 失败后仅修改测试而未修改生产代码的重复回合。
- 在 `scripts/`、`prototype/` 目录下批量生成测试文件。

不要把所有测试文件扩展名都 deny。硬约束应该和成本、风险及路径相关，而不是和文件后缀相关。

第四天：把“测试债务”单独观察

每周或合并后异步跑一次：全量测试、flaky 统计、覆盖率趋势、mutation sampling、E2E 和性能检查。它们的结果用于改进测试组合，不应成为 Agent 每次编辑的循环动作。

六、哪些方案看起来有效，实际会失败

失败方案 1：全局写“少写点测试”

模型无法把“少”映射成稳定动作。它不知道一个测试文件算多还是少，也不知道当前风险是否值得例外。结果通常是先写一堆测试，再在提示词里解释为什么合理。

失败方案 2：全局禁止 `*.test.*`

这会把低风险目录治理得很安静，却会让高风险变更失去回归保护；还可能迫使 Agent 把测试塞进生产文件或用脚本替代，反而降低可维护性。

失败方案 3：每次编辑都跑完整测试

这会把慢测试、环境依赖和 flaky 失败带进模型的即时反馈环。Agent 的修复回合会被 CI 延迟主导，且容易把环境问题误判成代码问题。

失败方案 4：把覆盖率设为任务验收标准

覆盖率只能告诉你哪些代码路径没有被触达，不能判断断言是否有意义。Google 的 change-detector 文章已经给出反例：测试可以覆盖 100% 的调用路径，却不验证任何正确行为。

失败方案 5：失败后继续补测试

一个无法定位的失败应先归类为代码失败、测试失败、环境失败或 flaky 失败。没有归因就继续加测试，只会扩大噪声和成本。

七、最终建议

最值得采用的不是“测试开关”，而是一个三段式默认策略：

即时阶段只做快速、可定位、受影响的验证；提交阶段做一次完整的相关检查；合并后异步做高保真和测试质量审计。

对 Agent 再加两条硬规则：

- 新增测试必须绑定一个明确的不变量、公共行为或回归缺陷；否则不写。
- 测试失败最多定向修复一次、重跑一次；仍不能归因就停止并汇报，不允许自发扩大测试范围。

这套方案既保留完整度，也保护迭代速度。完整度来自风险覆盖、受影响测试和发布级深测；速度来自快速反馈、限时执行、小型变更和可靠失败。测试数量只是结果，不应该成为模型的目标函数。

信息来源

1. Google Testing Blog, "Test Sizes", 2010-12-14. <https://testing.googleblog.com/2010/12/test-sizes.html>
2. Google Testing Blog, "Just Say No to More End-to-End Tests", 2015-04-20. <https://testing.googleblog.com/2015/04/just-say-no-to-more-end-to-end-tests.html>
3. Google Testing Blog, "Testing on the Toilet: Change Detector Tests", 2015-01-27. <https://testing.googleblog.com/2015/01/testing-on-toilet-change-detector-tests.html>
4. Google Testing Blog, "Mutation Testing", 2021-04-12. <https://testing.googleblog.com/2021/04/mutation-testing.html>
5. Google Testing Blog, "SMURF: Beyond the Test Pyramid", 2024-10-15. <https://testing.googleblog.com/2024/10/smurf-beyond-test-pyramid.html>
6. Google Testing Blog, "Test Failures Should Be Actionable", 2024-05-20. <https://testing.googleblog.com/2024/05/test-failures-should-be-actionable.html>
7. Google Testing Blog, "How I Learned To Stop Writing Brittle Tests and Love Expressive APIs", 2024-04-15. <https://testing.googleblog.com/2024/04/how-i-learned-to-stop-writing-brittle.html>
8. Google Testing Blog, "In Praise of Small Pull Requests", 2024-07-15. <https://testing.googleblog.com/2024/07/in-praise-of-small-pull-requests.html>
9. Martin Fowler, "The Practical Test Pyramid", 2018-02-26. <https://martinfowler.com/articles/practical-test-pyramid.html>
10. Aider Documentation, "Linting and testing". <https://aider.chat/docs/usage/lint-test.html>
11. Aider Documentation, "Configuration options". <https://aider.chat/docs/config/options.html>
12. Claude Code Documentation, "Configure permissions". <https://code.claude.com/docs/en/permissions>
13. Claude Code Documentation, "Hooks reference". <https://code.claude.com/docs/en/hooks>
14. GitHub Docs, "Adding repository custom instructions for GitHub Copilot". <https://docs.github.com/en/copilot/customizing-copilot/adding-repository-custom-instructions-for-github-copilot>
15. OpenAI Codex repository, `AGENTS.md`, commit `f73a07224653c2cc775b3f84f129b872b1e08f85`, 2026-07-08. <https://github.com/openai/codex/blob/main/AGENTS.md>

16. Goodenough, J. B. and Gerhart, S. L., “Toward a Theory of Test Data Selection”, 1975. <https://dl.acm.org/doi/10.1145/800027.808473>
17. Rothermel, G. and Harrold, M. J., “A Safe, Efficient Regression Test Selection Technique”, 1997. <https://doi.org/10.1145/248233.248262>
18. Bach, James, “Heuristic Risk-Based Testing”, 1999. <https://www.satisfice.us/articles/hrbt.pdf>
19. Martin Fowler, “Test Coverage”. <https://martinfowler.com/bliki/TestCoverage.html>
20. Martin Fowler, “Deployment Pipeline”. <https://martinfowler.com/bliki/DeploymentPipeline.html>
21. Meta Research, “Predictive Test Selection”, 2018. <https://research.facebook.com/publications/predictive-test-selection/>
22. Meta Engineering, “Predictive Test Selection”, 2018-11-21. <https://engineering.fb.com/2018/11/21/developer-tools/predictive-test-selection/>
23. Microsoft Learn, “Test Impact Analysis”. <https://learn.microsoft.com/en-us/azure/devops/pipelines/test/test-impact-analysis?view=azure-devops>
24. GitLab Documentation, `rules:changes`. <https://docs.gitlab.com/ci/yaml/#ruleschanges>
25. GitLab Documentation, “Merge trains”. https://docs.gitlab.com/ci/pipelines/merge_trains/
26. GitLab Development Guide, “Flaky tests”. https://docs.gitlab.com/development/testing_guide/flaky_tests/
27. Kubernetes Prow Documentation, “Jobs”. <https://docs.prow.k8s.io/docs/jobs/>
28. Google Testing Blog, “Flaky Tests at Google and How We Mitigate Them”, 2016-05-18. <https://testing.googleblog.com/2016/05/flaky-tests-at-google-and-how-we.html>
29. Bazel Documentation, “Test Encyclopedia”. <https://bazel.build/reference/test-encyclopedia>
30. GitHub Docs, “Managing a merge queue”. <https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/configuring-pull-request-merges/managing-a-merge-queue>

方法论说明

本报告采用横纵分析法：纵轴追踪测试工程从测试大小、测试金字塔到增量分析和 Agent 配置的演进；横轴比较 Google、OpenAI Codex、Aider、Claude Code、GitHub Copilot 与 Fowler 的当前机制，再交叉提炼可落地的 Agent 策略。